Find Duplicate Files

System Design & Interview Notes — single machine, incremental updates, and fleet-wide detection

Problem

Given a directory tree, find all sets of files with identical content. Then handle three follow-ups: how to optimize it, what happens when files change, and how to detect duplicates across many machines.

Two files are duplicates if their *bytes* are identical — name, path, timestamps and permissions are irrelevant. The naive approach ("hash every file, group by hash") is correct but reads every byte on disk, which is exactly the cost we want to avoid.

1. Baseline: The Filter Cascade

The core insight that carries the entire problem: apply a **cascade of progressively more expensive filters**, advancing only the groups that still contain two or more members. Singletons drop out at every stage, so the total bytes read ends up far below the total bytes on disk.

Stage	Cost	What it eliminates
1. Group by file size	`stat` only, no reads	~95%+ of files — a file with a unique size cannot have a duplicate
2. Group by (device, inode)	free from `stat`	Hardlinks — already the same physical file; report and skip
3. Hash first + last 4 KB	2 reads / file	Files differing in header or footer (media, documents, binaries)
4. Hash full content	full read	Everything else — this is the authoritative grouping
5. Byte-by-byte compare	full read ×2	Hash collisions — optional, only when exactness is mandatory

Reference implementation

```
// Stage 1: size buckets — metadata only, no file reads.
var bySize = new Dictionary<long, List<string>>();
foreach (var path in EnumerateFiles(root))        // iterative walk, not recursive
{
    var info = new FileInfo(path);
    if (info.Length == 0) continue;                // all empty files are trivially equal
    bySize.GetOrAdd(info.Length).Add(path);
}

var groups = new List<List<string>>();

// Only size buckets with >= 2 members can contain duplicates.
foreach (var candidates in bySize.Values.Where(g => g.Count > 1))
{
    // Stage 3: cheap prefix + suffix hash (2 reads per file).
    foreach (var partial in candidates.GroupBy(p => HashEdges(p, 4096))
                                      .Where(g => g.Count() > 1))
    {
        // Stage 4: full content hash, streamed in chunks (never load whole file).
        foreach (var full in partial.GroupBy(p => HashFull(p))
                                    .Where(g => g.Count() > 1))
            groups.Add(full.ToList());
    }
}
```

Complexity. `O(n)` syscalls for the walk, plus `O(bytes of surviving candidates)` of I/O — in practice a small fraction of the tree. Memory stays bounded by processing one size bucket at a time rather than materialising every hash at once.

Details interviewers probe for

- **Hash choice.** Use BLAKE3 or xxHash3, not MD5/SHA-1 — non-cryptographic hashes run roughly 10× faster. If the input is adversarial (user uploads), switch to a cryptographic hash: MD5 and SHA-1 collisions are trivially constructible today. With a 256-bit hash the birthday probability is negligible, which is why most production systems skip stage 5 entirely.
- **Symlinks.** Do not follow them by default, or you get directory cycles and phantom duplicates that are really the same file counted twice.
- **Parallelism.** The workload is I/O bound, so size the worker pool to the device: 32–64 concurrent readers on NVMe, but only 1–4 on spinning disks or you thrash the seek head and go slower than single-threaded. Hash computation itself parallelises freely across cores.
- **Empty files.** Every zero-byte file matches every other; filter them out rather than emitting one enormous useless group.
- **Sparse files and compressed filesystems.** Apparent size and allocated size differ; always bucket on logical (apparent) size.

2. What if files change?

Rescanning from scratch on every run is wasted work. Persist a **cache keyed by path** — this is precisely what `rsync`, `git` and Borg do:

```
path → (size, mtime_ns, ctime, inode, content_hash)
```

On the next run, `stat` each file and reuse the cached hash whenever `(size, mtime, inode)` is unchanged. Rehash only what actually moved. A rescan over millions of files then costs `O(n)` stats and close to zero I/O. SQLite is a good fit for this store: single file, transactional, indexed lookups.

Live updates

Subscribe to the platform's filesystem notification API — FSEvents on macOS, inotify on Linux, `ReadDirectoryChangesW` on Windows. Maintain an inverted index `hash → {paths}`. On a change event, remove the file's old hash entry, rehash it, and insert the new one, so duplicate groups stay correct *incrementally* instead of being recomputed wholesale.

> **Always pair watching with a periodic full rescan.** Watch queues have bounded size and drop events silently under load — inotify returns `IN_Q_OVERFLOW` and you lose an unknown set of changes. A nightly reconciliation pass is what keeps the index from drifting permanently out of sync.

Correctness hazards

- **TOCTOU race.** A file can be modified *while* you are hashing it, yielding a hash that matches no version that ever existed. Re-`stat` after hashing and discard the result if size or mtime moved. For a correctness-critical run, hash a filesystem snapshot (ZFS, LVM, VSS) instead of the live tree.
- **mtime is not trustworthy.** It is user-settable via `utimes()` and has coarse resolution on some filesystems, so "unchanged mtime" does not prove "unchanged content." Include `ctime` (inode change time, which is not settable) in the validity check, or schedule periodic verification rehashing.
- **Renames and moves.** Keying the cache on inode rather than path lets a moved file keep its cached hash instead of triggering a full rehash.

Files that change constantly

For actively-written files, whole-file dedup is the wrong tool — a single byte inserted invalidates the entire hash. Switch to **content-defined chunking**: split the file with a rolling hash (Rabin fingerprint) at content-determined boundaries, and dedup at chunk granularity. Because boundaries follow content rather than fixed offsets, an insertion in the middle only invalidates the one chunk containing it, instead of shifting every subsequent boundary. This is how Dropbox, restic and dedup storage arrays work.

3. Duplicates across many machines

The constraint inverts: the bottleneck is now **network**, not disk. The rule is that file contents never cross the wire. Each machine runs the local cascade and emits compact fingerprint records of roughly 50 bytes:

```
(content_hash, size, machine_id, path)
```

Two-phase protocol to cut traffic

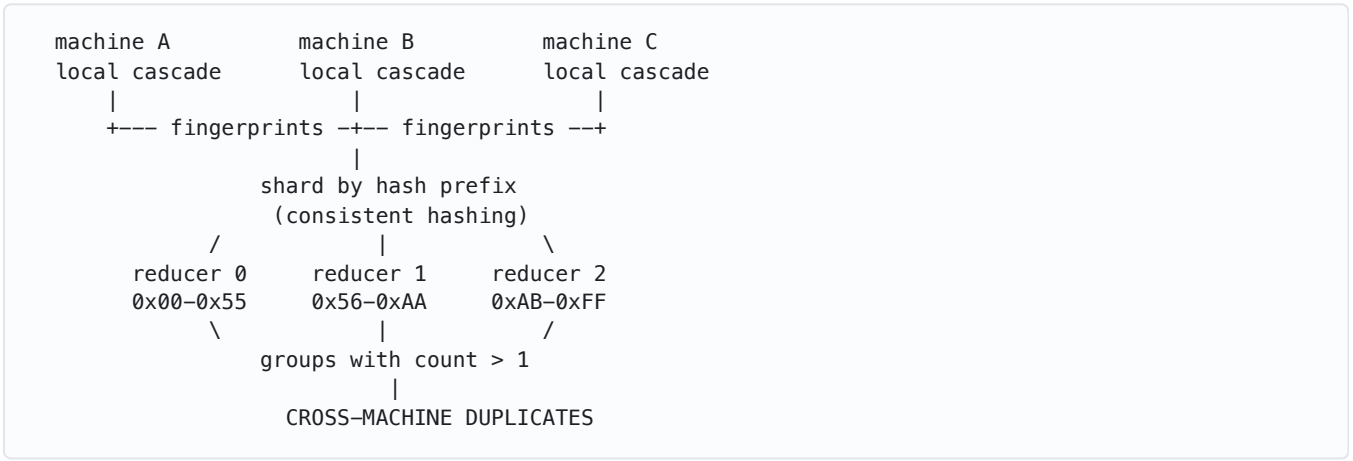
- **Round 1.** Every machine sends only its set of *distinct file sizes* — a tiny payload.
- **Round 2.** The coordinator returns the sizes observed on two or more machines. Machines then hash and report only files falling in those sizes.

On typical fleets this reduces the reported set by an order of magnitude, and — critically — each machine only reads the bytes that could possibly participate in a cross-machine duplicate.

Aggregation

What remains is a distributed group-by, which is embarrassingly parallel:

- Shard by **hash prefix** using consistent hashing, so every copy of a given hash lands on the same reducer regardless of which machine it came from.
- In MapReduce/Spark form: `map` emits `hash → record`, `reduce` keeps groups of size > 1.
- Scale check: 1 billion files × 50 B = ~50 GB of fingerprints — comfortably a distributed sort, handled by a handful of nodes.



Operational concerns

- **Incremental sync.** Each machine keeps the local cache from section 2 and pushes only deltas — added, changed, deleted — since its last checkpoint. Steady-state traffic then scales with churn, not with fleet size.
- **Deletions and dead machines.** Use tombstones with generation numbers, or scope each machine's records under a lease that expires without heartbeats. Without this you accumulate phantom duplicates pointing at files that no longer exist.
- **Verification is expensive.** Byte-comparing across machines means transferring an entire file. With a 256-bit hash, treat hash equality as identity — the standard tradeoff every production dedup system makes.
- **Stragglers.** One machine holding 10M files should not block the whole report. Make aggregation incremental and queryable while still partial.
- **Consistency model.** The result is eventually consistent by nature: files change during the scan, so the report describes a fuzzy snapshot, not a point-in-time truth. Say this out loud rather than pretending otherwise.

Summary

Scenario	Key technique	Dominant cost
Single directory	Size → edge-hash → full-hash cascade	I/O on surviving candidates only
Repeated scans	Persistent cache keyed on (size, mtime, inode)	`O(n)` stat calls
Live filesystem	Watch API + inverted index + periodic reconcile	Proportional to churn
Changing files	Content-defined chunking (Rabin fingerprint)	Per-chunk, not per-file
Many machines	Two-phase fingerprint exchange, shard by hash	Network, ~50 B per file

> **Where to start in an interview.** Lead with the size → partial-hash → full-hash cascade, because that single insight carries the whole problem. The caching layer and the distributed version are both just the same cascade with work you have already done skipped — frame them that way and the follow-ups answer themselves.